Engine de traitement et d'analyse de flux de logs

Projet final Python — ING 3 , ECE Paris

Équipe : Joan MBALLA, BAYA Youness, PHILIPPE Mathis

Dépôt : `log-engine` — rendu du lundi 9 novembre 2026

Ce rapport illustre le fonctionnement du projet. Chaque section correspond à un critère du barème, et toutes les sorties viennent de l'exécution du vrai code du dépôt, pas de copier-coller.

#	Section	Barème
1	Environnement et livrables	2 pts
2	Gestion mémoire et générateurs	4 pts
3	Structures de données	4 pts
4	Programmation fonctionnelle	4 pts
5	Moteur de règles et introspection	3 pts
6	Qualité du code et robustesse	3 pts
7	Résultats	—
8	Répartition du travail	—

Architecture du pipeline

```
data/*.log
  │ M1  read_multiple_logs → read_logs → parse_logs
  ▼
LogEntry | None
  │ M2  filter_entries → normalize → route_all
  ▼
TransformedEntry
  ├─ M4  apply_security_rules(*règles, **config)   passe 1 : alertes
  └─ M3  analyze                                   passe 2 : agrégats
  ▼
Audit globals() / locals() → JSON                  M4
```

Chaque étape reçoit et rend un itérable : aucune étape ne construit la liste complète des logs.

In [1]:
```
import inspect
import os
import sys
from pathlib import Path

from IPython.display import Markdown, display

root = Path.cwd()
if not (root / "log_engine").exists():
    root = root.parent
os.chdir(root)
sys.path.insert(0, str(root))


def show(*objects):
```

```
    for obj in objects:
        display(Markdown(f"```python\n{inspect.getsource(obj)}```"))


print("Racine du projet :", root.name)
```

```
Racine du projet : log-engine
```

1 . Environnement et livrables (2 pts)

Le projet tourne dans un environnement virtuel `venv` isolé :

```
python3 -m venv .venv
source .venv/bin/activate          # Windows : .venv\Scripts\activate
pip install -r requirements.txt
python main.py                     # pipeline complet sur data/
python main.py --audit audit.json  # + export du rapport d'audit
python -m unittest -v              # 79 tests
```

`requirements.txt` a été généré avec `pip freeze` dans le venv du projet. Il est vide (à part un commentaire) parce que le package n'utilise que la bibliothèque standard : il n'y a aucune dépendance externe à installer.

La cellule suivante le vérifie : elle parcourt tous les `import` du package et de `main.py` avec `ast`, et confirme que chaque module appartient à la bibliothèque standard ou au package lui-même.

In [2]:
```
import ast

print("requirements.txt :")
print(Path("requirements.txt").read_text(encoding="utf-8"))

sources = sorted(Path("log_engine").glob("*.py")) + [Path("main.py")]
imported = set()
for path in sources:
    tree = ast.parse(path.read_text(encoding="utf-8"))
    for node in ast.walk(tree):
        if isinstance(node, ast.Import):
            imported |= {alias.name.split(".")[0] for alias in node.names}
        elif isinstance(node, ast.ImportFrom) and node.module and node.level == 0:
            imported.add(node.module.split(".")[0])

external = imported - set(sys.stdlib_module_names) - {"log_engine", "__future__"}
print("Modules importés :", sorted(imported))
print("Dépendances externes :", sorted(external) or "aucune")
```

```
requirements.txt :
# Généré via pip freeze : stdlib uniquement, aucune dépendance externe.

Modules importés : ['__future__', 'argparse', 'collections', 'dataclasses', 'datetime',
'itertools', 'json', 'log_engine', 'pathlib', 'sys', 'types', 'typing']
Dépendances externes : aucune
```

Arborescence du dépôt : le code est un package Python (`log_engine/`), les notebooks servent uniquement au rapport, et `main.py` pilote le pipeline.

In [3]:
```
for path in sorted(Path(".").rglob("*")):
    parts = path.parts
    if any(p.startswith(".") or p == "__pycache__" for p in parts):
        continue
    if path.is_file():
        print("  " * (len(parts) - 1) + path.name)
    else:
        print("  " * (len(parts) - 1) + path.name + "/")
```

```
README.md
data/
  server_a.log
  server_b.log
log_engine/
  __init__.py
  aggregation.py
  ingestion.py
  rules.py
  transform.py
main.py
notebooks/
  rapport.ipynb
requirements.txt
tests/
  __init__.py
  test_aggregation.py
  test_ingestion.py
  test_rules.py
  test_transform.py
```

2 . Gestion mémoire et générateurs (4 pts)

`read_logs` : lecture ligne par ligne avec `yield`

Le fichier est ouvert puis parcouru ligne par ligne. Chaque ligne est rendue avec `yield` au moment où le consommateur la demande : il n'y a jamais plus d'une ligne en mémoire.

`read_multiple_logs` : enchaînement de fichiers avec `yield from`

`yield from` délègue à `read_logs` pour chaque fichier, l'un après l'autre. Un fichier absent est signalé puis ignoré (`continue`) sans arrêter le flux.

In [4]:
```
from log_engine.ingestion import parse_logs, read_logs, read_multiple_logs

show(read_logs, read_multiple_logs, parse_logs)
```

```
def read_logs(path: str | Path, encoding: str = "utf-8") -> Iterator[str]:
    """Lit un fichier ligne par ligne, sans jamais le charger en entier."""
    with open(path, encoding=encoding) as handle:
        for line in handle:
            stripped = line.rstrip("\r\n")
            if stripped.strip():
                yield stripped

def read_multiple_logs(paths: Iterable[str | Path]) -> Iterator[str]:
    """Enchaîne plusieurs fichiers de logs, un par un."""
    for path in paths:
        try:
            yield from read_logs(path)
        except FileNotFoundError:
            print(f"[ingestion] fichier introuvable, ignoré : {path}")
            continue

def parse_logs(lines: Iterable[str]) -> Iterator[LogEntry | None]:
    """Applique parse_line paresseusement sur un flux de lignes."""
    for line in lines:
        yield parse_line(line)
```

In [5]:
```
from types import GeneratorType

paths = sorted(Path("data").glob("*.log"))
flux = read_multiple_logs(paths)
print("Type :", type(flux).__name__, "| générateur :", isinstance(flux, GeneratorType))
```

```
print("1re ligne :", next(flux))
print("2e ligne  :", next(flux))
flux.close()
```

```
Type : generator | générateur : True
1re ligne : 2026-11-03T08:00:01 | INFO | 192.168.1.10:443/TCP | alice | {"action": "login",
"status": "success"}
2e ligne  : 2026-11-03T08:00:05 | INFO | 192.168.1.11:443/TCP | bob | {"action": "view", "page":
"/dashboard"}
```

Mesure : générateur contre liste

On génère un fichier de 200 000 lignes, puis on mesure le pic mémoire avec `tracemalloc` dans deux cas : en parcourant le générateur, et en chargeant tout le fichier dans une liste. On mesure aussi le pipeline complet (lecture, parsing, transformation) sur ce même fichier.

In [6]:
```
import tempfile
import tracemalloc

from log_engine.transform import transform

LIGNE = ('2026-11-03T08:00:01 | INFO | 192.168.1.10:443/TCP | alice | '
         '{"action": "login", "status": "success"}\n')
N = 200_000

tmp_dir = tempfile.TemporaryDirectory()
big = Path(tmp_dir.name) / "big.log"
with open(big, "w", encoding="utf-8") as handle:
    for _ in range(N):
        handle.write(LIGNE)


def pic_memoire(fonction):
    tracemalloc.start()
    resultat = fonction()
    _, pic = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return resultat, pic


n_gen, pic_gen = pic_memoire(lambda: sum(1 for _ in read_logs(big)))
n_list, pic_list = pic_memoire(lambda: len(list(read_logs(big))))
n_pipe, pic_pipe = pic_memoire(
    lambda: sum(1 for _ in transform(parse_logs(read_logs(big))))
)

print(f"Fichier : {N:,} lignes, {big.stat().st_size / 1e6:.1f} Mo sur disque")
print(f"Générateur      : {n_gen:,} lignes, pic {pic_gen / 1e3:9.1f} Ko")
print(f"Liste complète  : {n_list:,} lignes, pic {pic_list / 1e3:9.1f} Ko")
print(f"Pipeline 1 → 2  : {n_pipe:,} entrées, pic {pic_pipe / 1e3:9.1f} Ko")
print(f"La liste consomme {pic_list / pic_gen:.0f} fois plus de mémoire que le générateur.")
tmp_dir.cleanup()
```

```
Fichier : 200,000 lignes, 20.2 Mo sur disque
Générateur      : 200,000 lignes, pic      22.8 Ko
Liste complète  : 200,000 lignes, pic   29838.1 Ko
Pipeline 1 → 2  : 200,000 entrées, pic      33.7 Ko
La liste consomme 1311 fois plus de mémoire que le générateur.
```

Le pic du générateur ne dépend pas de la taille du fichier : il reste de l'ordre de quelques kilo-octets, alors que la liste grossit avec le nombre de lignes. Le pipeline complet reste lui aussi borné, parce que `filter` et `map` sont paresseux.

`main.py` fait deux passes, toujours en flux. La passe 1 applique les règles du module 4 et affiche les alertes entrée par entrée. La passe 2 relit les fichiers pour l'agrégation du module 3. On relit le disque plutôt que de stocker les entrées de

la passe `1` dans une liste : c'est un choix délibéré, qui garde une mémoire constante au prix d'une seconde lecture.

3 . Structures de données (4 pts)

Tuples : métadonnées immuables `(ip, port, protocole)`

`parse_line` range les métadonnées fixes de connexion dans un tuple. Comme le tuple est immuable, aucun module ne peut les modifier par erreur.

In [7]:
```
from log_engine.ingestion import parse_line

ligne = ('2026-11-03T08:01:10 | WARNING | 10.0.0.5:22/TCP | root | '
         '{"action": "login", "status": "failed"}')
entry = parse_line(ligne)
for cle, valeur in entry.items():
    print(f"{cle:<10} {type(valeur).__name__:<9} {valeur!r}")

try:
    entry["meta"][1] = 2222
except TypeError as exc:
    print("\nModification de meta refusée :", exc)
```

```
timestamp  datetime  datetime.datetime(2026, 11, 3, 8, 1, 10)
level      str       'WARNING'
ip         str       '10.0.0.5'
user       str       'root'
payload    dict      {'action': 'login', 'status': 'failed'}
meta       tuple     ('10.0.0.5', 22, 'TCP')

Modification de meta refusée : 'tuple' object does not support item assignment
```

`set` et `frozenset` : IP uniques et blacklist

Le module `3` accumule les IP actives et les IP en erreur dans des `set`. La blacklist est un `frozenset` immuable. `compare_with_blacklist` applique les quatre opérations ensemblistes demandées : union `|`, intersection `&`, différence `-` et différence symétrique `^`.

In [8]:
```
from log_engine.aggregation import DEFAULT_BLACKLIST, analyze, compare_with_blacklist

show(compare_with_blacklist)
```

```
def compare_with_blacklist(
    active_ips: Iterable[str],
    blacklist: frozenset[str] = DEFAULT_BLACKLIST,
) -> SetComparison:
    """Applique ``|``, ``&``, ``-`` et ``^`` entre IP actives et blacklist."""
    active = frozenset(active_ips)
    return SetComparison(
        union=active | blacklist,
        intersection=active & blacklist,
        only_active=active - blacklist,
        only_blacklist=blacklist - active,
        symmetric_difference=active ^ blacklist,
    )
```

In [9]:
```
rapport = analyze(transform(parse_logs(read_multiple_logs(paths))))

print("IP en erreur (set)  :", sorted(rapport.error_ips))
print("IP actives (set)    :", sorted(rapport.active_ips))
print("Blacklist (frozenset):", sorted(DEFAULT_BLACKLIST))
```

```
print()
for nom, valeur in rapport.blacklist._asdict().items():
    print(f"{nom:<22} {sorted(valeur)}")

try:
    DEFAULT_BLACKLIST.add("1.2.3.4")
except AttributeError as exc:
    print("\nAjout à la blacklist refusé :", exc)
```

```
IP en erreur (set)  : ['10.0.0.5', '172.16.0.3', '203.0.113.7']
IP actives (set)    : ['10.0.0.5', '172.16.0.3', '192.168.1.10', '192.168.1.11', '192.168.1.12',
'192.168.1.13', '198.51.100.4', '203.0.113.7']
Blacklist (frozenset): ['198.51.100.4', '203.0.113.7', '45.33.32.156']

union                 ['10.0.0.5', '172.16.0.3', '192.168.1.10', '192.168.1.11',
'192.168.1.12', '192.168.1.13', '198.51.100.4', '203.0.113.7', '45.33.32.156']
intersection          ['198.51.100.4', '203.0.113.7']
only_active           ['10.0.0.5', '172.16.0.3', '192.168.1.10', '192.168.1.11',
'192.168.1.12', '192.168.1.13']
only_blacklist        ['45.33.32.156']
symmetric_difference  ['10.0.0.5', '172.16.0.3', '192.168.1.10', '192.168.1.11',
'192.168.1.12', '192.168.1.13', '45.33.32.156']

Ajout à la blacklist refusé : 'frozenset' object has no attribute 'add'
```

Lecture du résultat avec la blacklist par défaut du module 3 : deux IP blacklistées (`203.0.113.7` et `198.51.100.4`) sont actives dans nos logs (intersection), et `45.33.32.156` est blacklistée mais n'apparaît pas (différence `blacklist - actives`).

`defaultdict` : comptage sans `KeyError`

`defaultdict(int)` compte les entrées par canal, par utilisateur et par niveau : la première occurrence d'une clé démarre à 0 , donc aucune `KeyError` n'est possible. Le rapport convertit ensuite les compteurs en `dict` simples, pour qu'une lecture de clé absente ne crée pas de clé par accident.

In [10]:
```
from collections import defaultdict

print("Par canal  :", rapport.by_channel)
print("Par user   :", rapport.by_user)
print("Par niveau :", rapport.by_level)

compteur = defaultdict(int)
compteur["jamais_vu"] += 1
print("\ndefaultdict sur une clé inconnue :", dict(compteur))
```

```
Par canal  : {'auth': 2, 'event': 2, 'auth_failure': 5, 'error': 3, 'raw': 3}
Par user   : {'alice': 2, 'bob': 1, 'root': 5, 'carol': 1, 'dave': 2, 'admin': 2, 'guest': 1,
'erin': 1}
Par niveau : {'INFO': 6, 'WARNING': 4, 'ERROR': 4, 'CRITICAL': 1}

defaultdict sur une clé inconnue : {'jamais_vu': 1}
```

`match-case` : routage selon le format

`route` (module 2) remplace une cascade de `if-elif-else` par du Structural Pattern Matching. Elle reconnaît les trois formats de l'énoncé : chaîne brute, dictionnaire du type `{'level': 'ERROR', 'code': code}` et tuple de connexion. Les motifs vérifient à la fois la forme et les types (`int()` , `str()`), et capturent les valeurs utiles avec `as` .

In [11]:
```
from log_engine.transform import route

show(route)
```

```python
def route(record: object) -> tuple[str, Any]:
    """Classe un enregistrement selon son format avec match-case."""
    match record:
        case (str() as ip, int() as port, str() as protocol):
            return ("connection", f"{ip}:{port}/{protocol}")
        case {"level": "ERROR" | "CRITICAL" as level, "code": int() as code}:
            return ("error", f"{level} {code}")
        case {"action": "login", "status": "failed"}:
            return ("auth_failure", "login failed")
        case {"action": "login", "status": str() as status}:
            return ("auth", f"login {status}")
        case dict():
            return ("event", record)
        case str() if record.strip():
            return ("raw", record)
        case _:
            return ("unknown", record)
```

In [12]:

```python
exemples = [
    "DNS query example.com",
    {"level": "ERROR", "code": 500},
    {"level": "CRITICAL", "code": 1045, "message": "Access denied"},
    {"action": "login", "status": "failed"},
    {"action": "login", "status": "success"},
    {"action": "logout"},
    ("10.0.0.5", 22, "TCP"),
    ("10.0.0.5", "22", "TCP"),
    "   ",
    42,
]
for record in exemples:
    canal, detail = route(record)
    print(f"{canal:<13} ← {record!r}")
```

```
raw           ← 'DNS query example.com'
error         ← {'level': 'ERROR', 'code': 500}
error         ← {'level': 'CRITICAL', 'code': 1045, 'message': 'Access denied'}
auth_failure  ← {'action': 'login', 'status': 'failed'}
auth          ← {'action': 'login', 'status': 'success'}
event         ← {'action': 'logout'}
connection    ← ('10.0.0.5', 22, 'TCP')
unknown       ← ('10.0.0.5', '22', 'TCP')
unknown       ← '   '
unknown       ← 42
```

4 . Programmation fonctionnelle (4 pts)

`filter()` et `lambda`

`filter_entries` enchaîne deux filtres. `filter(None, ...)` retire les `None` que `parse_logs` laisse passer pour les lignes invalides. Ensuite, un `filter` avec une `lambda` écarte les entrées sans utilisateur, sans IP ou avec un niveau inconnu.

`map()`

`normalize` applique `normalize_entry` à chaque entrée avec `map`. Elle met `user` en minuscules et `level` en majuscules, et réduit les espaces en trop dans les chaînes. Dans un payload dict, les clés passent en minuscules. Elle ajoute aussi un résumé construit avec une f-string. La fonction renvoie une copie et ne modifie jamais l'entrée d'origine. `route_all` utilise aussi `map` pour appliquer le routage.

In [13]:
```
from log_engine.transform import filter_entries, normalize, normalize_entry, route_all

show(filter_entries, normalize_entry, normalize, route_all, transform)
```

```
def filter_entries(entries: Iterable[LogEntry | None]) -> Iterator[LogEntry]:
    """Retire les None puis les entrées sans utilisateur ou sans IP."""
    parsed = filter(None, entries)
    return filter(
        lambda e: bool(e["user"]) and bool(e["ip"]) and e["level"] in LEVELS,
        parsed,
    )

def normalize_entry(entry: LogEntry) -> TransformedEntry:
    """Renvoie une copie nettoyée de l'entrée, avec un résumé lisible."""
    raw_ip, port, raw_protocol = entry["meta"]
    ip = raw_ip.strip()
    protocol = raw_protocol.strip().upper()
    user = entry["user"].strip().lower()
    level = entry["level"].strip().upper()
    normalized: TransformedEntry = {
        "timestamp": entry["timestamp"],
        "level": level,
        "ip": ip,
        "user": user,
        "payload": _normalize_payload(entry["payload"]),
        "meta": (ip, port, protocol),
    }
    normalized["summary"] = (
        f"[{entry['timestamp']:%Y-%m-%d %H:%M:%S}] {level:<8} "
        f"{user}@{ip}:{port}/{protocol}"
    )
    return normalized

def normalize(entries: Iterable[LogEntry]) -> Iterator[TransformedEntry]:
    """Applique normalize_entry à chaque entrée avec map()."""
    return map(normalize_entry, entries)

def route_all(
    entries: Iterable[TransformedEntry],
) -> Iterator[TransformedEntry]:
    """Applique route_entry à chaque entrée avec map()."""
    return map(route_entry, entries)

def transform(
    entries: Iterable[LogEntry | None],
) -> Iterator[TransformedEntry]:
    """Chaîne complète du module 2 : filtre, normalisation, routage."""
    return route_all(normalize(filter_entries(entries)))
```

In [14]:
```
brutes = list(parse_logs(read_multiple_logs(paths)))
valides = list(filter_entries(brutes))
print(f"Lignes parsées : {len(brutes)} | None : {brutes.count(None)} | après filter :
{len(valides)}")

sale = {
    **parse_line(ligne),
    "user": "  ROOT ",
    "payload": {" Action ": " LOGIN ", "Status": "FAILED"},
}
propre = normalize_entry(sale)
print("\nAvant :", sale["user"], sale["payload"])
print("Après :", propre["user"], propre["payload"])
print("Résumé (f-string) :", propre["summary"])
print("Entrée d'origine intacte :", sale["user"] == "  ROOT ")

paresseux = transform(iter(brutes))
print("\ntransform() rend :", type(paresseux).__name__, "→ rien n'est calculé avant next()")
```

```
Lignes parsées : 19 | None : 4 | après filter : 15

Avant :   ROOT  {' Action ': ' LOGIN ', 'Status': 'FAILED'}
Après : root {'action': 'login', 'status': 'failed'}
Résumé (f-string) : [2026-11-03 08:01:10] WARNING  root@10.0.0.5:22/TCP
Entrée d'origine intacte : True

transform() rend : map → rien n'est calculé avant next()
```

Ici, `list()` sert uniquement à compter pour la démonstration. Dans le pipeline, rien n'est matérialisé.

Fonctions passées en argument

Les règles de sécurité du module 4 sont des fonctions ordinaires, manipulées comme des objets de première classe : on les passe à `apply_security_rules`, qui les appelle sur chaque entrée. N'importe quelle fonction respectant la signature `regle(entry, **config) -> str | None` est acceptée, y compris une `lambda` écrite au moment de l'appel.

In [15]:
```python
from log_engine.rules import apply_security_rules, rule_auth_failure, rule_severity

regle_root = lambda e, **_: "root_account" if e.get("user") == "root" else None

resultats = apply_security_rules(
    rule_severity,
    rule_auth_failure,
    regle_root,
    entries=transform(parse_logs(read_multiple_logs(paths))),
    only_alerts=True,
)
for e in resultats:
    print(f"{e['summary']}  {e['alerts']}")
```

```
[2026-11-03 08:01:10] WARNING  root@10.0.0.5:22/TCP  ('auth_failure:10.0.0.5', 'root_account')
[2026-11-03 08:01:12] WARNING  root@10.0.0.5:22/TCP  ('auth_failure:10.0.0.5', 'root_account')
[2026-11-03 08:01:14] WARNING  root@10.0.0.5:22/TCP  ('auth_failure:10.0.0.5', 'root_account')
[2026-11-03 08:01:16] ERROR    root@10.0.0.5:22/TCP  ('severity:ERROR', 'auth_failure:10.0.0.5',
'root_account')
[2026-11-03 08:01:18] CRITICAL root@10.0.0.5:22/TCP  ('severity:CRITICAL', 'auth_failure:
10.0.0.5', 'root_account')
[2026-11-03 08:02:00] ERROR    carol@172.16.0.3:8080/HTTP  ('severity:ERROR',)
[2026-11-03 09:01:00] ERROR    admin@203.0.113.7:3306/TCP  ('severity:ERROR',)
[2026-11-03 09:01:05] ERROR    admin@203.0.113.7:3306/TCP  ('severity:ERROR',)
```

5 . Moteur de règles et introspection (3 pts)

`apply_security_rules(*args, **kwargs)`

- `*args` reçoit un nombre variable de règles. Sans règle fournie, le moteur découvre toutes les fonctions `rule_*` du module grâce à `globals()`.
- `**kwargs` reçoit le flux (`entries=`), deux options du moteur (`only_alerts` , `strict`) et tout le reste comme configuration dynamique transmise aux règles. Par exemple : `blacklist` , `sensitive_ports` , `work_hours` , `patterns` .
- Le moteur rend un générateur. Les erreurs d'arguments sont levées dès l'appel, parce que le générateur interne `_run_rules` est séparé de la fonction publique.
- Si une règle plante, l'erreur est notée dans les alertes et le traitement continue. Avec `strict=True` , elle remonte.

In [16]:
```python
```

```
from log_engine.rules import discover_rules

show(apply_security_rules, discover_rules)
```

```
def apply_security_rules(*args: Rule,
                         **kwargs: Any) -> Iterator[TransformedEntry]:
    """Applique les règles reçues en ``*args`` à chaque entrée.

    Arguments nommés reconnus (le reste est transmis tel quel aux règles) :
        entries      : itérable de LogEntry (obligatoire).
        only_alerts  : si True, ne rend que les entrées avec au moins
                       une alerte (défaut False).
        strict       : si True, une règle qui lève une exception arrête
                       le pipeline ; sinon l'erreur est notée dans les
                       alertes et on continue (défaut False).

    Sans règle passée en ``*args``, toutes les règles ``rule_*`` du
    module sont utilisées.

    Rend un générateur de nouveaux dicts (l'entrée d'origine n'est pas
    modifiée) enrichis d'une clé ``"alerts"`` (tuple de libellés).
    """
    if "entries" not in kwargs:
        raise TypeError("apply_security_rules() requiert entries=<itérable>")
    entries: Iterable[LogEntry] = kwargs.pop("entries")
    only_alerts: bool = kwargs.pop("only_alerts", False)
    strict: bool = kwargs.pop("strict", False)
    config = kwargs  # ce qui reste = configuration dynamique des règles

    not_callable = [repr(a) for a in args if not callable(a)]
    if not_callable:
        raise TypeError(f"Règles non appelables : {', '.join(not_callable)}")
    rules = args or discover_rules()

    return _run_rules(entries, rules, config, only_alerts, strict)
```

```
def discover_rules(namespace: Optional[Mapping[str, Any]] = None,
                   prefix: str = RULE_PREFIX) -> tuple[Rule, ...]:
    """Découvre dynamiquement les règles via ``globals()``.

    Toute fonction dont le nom commence par ``prefix`` est une règle.
    On peut passer un autre espace de noms (ex. ``globals()`` de main.py)
    pour y découvrir des règles ajoutées ailleurs.
    """
    scope = globals() if namespace is None else namespace
    return tuple(
        obj for name, obj in sorted(scope.items())
        if name.startswith(prefix) and callable(obj)
    )
```

In [17]:

```
print("Règles découvertes via globals() :")
for regle in discover_rules():
    print("  -", regle.__name__)

flux = apply_security_rules(
    entries=transform(parse_logs(read_multiple_logs(paths))),
    blacklist=frozenset({"10.0.0.5"}),
    sensitive_ports=frozenset({22, 3306}),
    only_alerts=True,
)
print("\nToutes les règles, ports sensibles configurés à {22, 3306} :")
for e in flux:
    print(f"  {e['ip']:<12} {e['alerts']}")


def regle_bugguee(entry, **_):
    return entry["cle_absente"]
```

```
tolerant = next(apply_security_rules(regle_bugguee, entries=[valides[0]]))
print("\nRègle qui plante, mode tolérant :", tolerant["alerts"])
try:
    next(apply_security_rules(regle_bugguee, entries=[valides[0]], strict=True))
except KeyError as exc:
    print("Mode strict : KeyError", exc)
```

```
Règles découvertes via globals() :
  - rule_auth_failure
  - rule_blacklisted_ip
  - rule_off_hours
  - rule_sensitive_port
  - rule_severity
  - rule_suspicious_payload

Toutes les règles, ports sensibles configurés à {22, 3306} :
  10.0.0.5     ('auth_failure:10.0.0.5', 'blacklisted_ip:10.0.0.5', 'sensitive_port:22/TCP')
  10.0.0.5     ('auth_failure:10.0.0.5', 'blacklisted_ip:10.0.0.5', 'sensitive_port:22/TCP')
  10.0.0.5     ('auth_failure:10.0.0.5', 'blacklisted_ip:10.0.0.5', 'sensitive_port:22/TCP')
  10.0.0.5     ('auth_failure:10.0.0.5', 'blacklisted_ip:10.0.0.5', 'sensitive_port:22/TCP',
'severity:ERROR')
  10.0.0.5     ('auth_failure:10.0.0.5', 'blacklisted_ip:10.0.0.5', 'sensitive_port:22/TCP',
'severity:CRITICAL')
  172.16.0.3   ('severity:ERROR',)
  203.0.113.7  ('sensitive_port:3306/TCP', 'severity:ERROR')
  203.0.113.7  ('sensitive_port:3306/TCP', 'severity:ERROR')

Règle qui plante, mode tolérant : ('rule_error:regle_bugguee:KeyError',)
Mode strict : KeyError 'cle_absente'
```

Audit `globals()` / `locals()`

`audit_scope` inspecte un espace de noms : pour chaque nom, elle relève sa catégorie (module, classe, fonction, variable), son type, sa taille (`sys.getsizeof`) et un aperçu tronqué. Elle travaille sur une copie de l'espace de noms et n'appelle jamais `next()` sur un générateur, donc l'audit ne consomme pas le flux. `audit_runtime(globals(), locals())` produit le rapport des deux portées et `export_audit_report` l'écrit en JSON.

In [18]:
```
import json

from log_engine.rules import audit_runtime, export_audit_report, format_audit_report


def etape_de_traitement():
    seuil = 5
    ips_suspectes = {"10.0.0.5"}
    flux_local = transform(parse_logs(read_multiple_logs(paths)))
    return audit_runtime(globals(), locals())


audit = etape_de_traitement()
print(format_audit_report(audit))
print("\nPortée locale de etape_de_traitement() :")
for var in audit["locals"]["variables"]:
    print(f"  {var['name']:<14} {var['type']:<10} {var['size_bytes']:>5} o
{var['preview']}")

with tempfile.TemporaryDirectory() as tmp:
    chemin = export_audit_report(audit, Path(tmp) / "audit.json")
    relu = json.loads(chemin.read_text(encoding="utf-8"))
print("\nJSON exporté puis relu : clés =", list(relu))
```

```
Audit du 2026-11-08T17:07:20 (Python 3.12.3)
  globals : 84 noms (class=4, function=30, module=7, variable=43), 15717 octets
  locals  : 3 noms (variable=3), 292 octets

Portée locale de etape_de_traitement() :
  flux_local     map           48 o  <map object at 0x73bf90e88ee0>
  ips_suspectes  set          216 o  {'10.0.0.5'}
  seuil          int           28 o  5

JSON exporté puis relu : clés = ['generated_at', 'python', 'globals', 'locals']
```

6 . Qualité du code et robustesse (3 pts)

PEP 8 . `pycodestyle` passe sans aucune erreur sur `log_engine/` , `tests/` et `main.py` .

Typage. Toutes les fonctions publiques sont annotées. Les structures de données sont typées : `TypedDict` pour `LogEntry` et `TransformedEntry` , `NamedTuple` pour `SetComparison` , `dataclass` pour `BruteForceAlert` et `AnalysisReport` , et `tuple[str, int, str]` pour `meta` .

Exceptions.

- `parse_line` attrape les `ValueError` (date ou port invalide) et rend `None` au lieu de planter.
- Un JSON cassé dans le payload est conservé en texte brut (`json.JSONDecodeError`).
- Un fichier absent est signalé et ignoré (`FileNotFoundError`).
- Le moteur de règles isole les erreurs des règles (mode tolérant ou `strict`).
- `analyze` valide ses paramètres (`ValueError` si le seuil est inférieur à 1).

In [19]:
```python
lignes_invalides = [
    "ligne corrompue sans aucun séparateur",
    "2026-11-03T09:00:30 | DEBUG | 192.168.1.12:443/TCP | dave | niveau inconnu",
    "2026-13-45T99:00:00 | INFO | 192.168.1.12:443/TCP | dave | date invalide",
    "2026-11-03T09:04:00 | CRITICAL | 198.51.100.4:abc/TCP | guest | port invalide",
]
for l in lignes_invalides:
    print(f"{str(parse_line(l)):<5} ← {l}")

cassee = parse_line("2026-11-03T09:02:00 | WARNING | 198.51.100.4:80/HTTP | guest | {payload
json cassé")
print("\nJSON cassé conservé en texte :", repr(cassee["payload"]))

print()
print(sum(1 for _ in read_multiple_logs([paths[0], Path("data/absent.log")])), "lignes lues
malgré le fichier absent")

try:
    analyze([], threshold=0)
except ValueError as exc:
    print("analyze(threshold=0) :", exc)
```

```
None  ← ligne corrompue sans aucun séparateur
None  ← 2026-11-03T09:00:30 | DEBUG | 192.168.1.12:443/TCP | dave | niveau inconnu
None  ← 2026-13-45T99:00:00 | INFO | 192.168.1.12:443/TCP | dave | date invalide
None  ← 2026-11-03T09:04:00 | CRITICAL | 198.51.100.4:abc/TCP | guest | port invalide

JSON cassé conservé en texte : '{payload json cassé'

[ingestion] fichier introuvable, ignoré : data/absent.log
10 lignes lues malgré le fichier absent
analyze(threshold=0) : threshold doit être >= 1
```

Boucles : `while`, `for`, `break`, `continue` et `else`

La détection de force brute (module 3) combine les deux boucles :

- `while True` découpe le flux en blocs de taille bornée avec `islice`, et sort par `break` quand le flux est épuisé.
- `for` parcourt un bloc. `continue` passe les événements qui ne sont pas des connexions, et les connexions réussies, qui remettent la série de l'IP à zéro.
- Le `else` du `for` s'exécute seulement si le bloc a été traité **jusqu'au bout**, sans `break`. Il compte alors un cycle de détection réussi.
- Si le nombre maximal d'alertes est atteint, le `break` du `for` saute le `else` : l'analyse s'arrête tôt et `completed` passe à `False`.

Règle de détection : 5 échecs de connexion consécutifs depuis la même IP.

In [20]:
```
src = inspect.getsource(analyze)
debut = src.index("    while True:")
fin = src.index("    return AnalysisReport(")
display(Markdown(f"```python\n{src[debut:fin]}```"))
```

```python
    while True:
        chunk = tuple(islice(iterator, chunk_size))
        if not chunk:
            break  # flux épuisé
        for entry in chunk:
            scanned += 1
            channel = entry.get("channel", "unknown")
            ip = entry["ip"]
            by_channel[channel] += 1
            by_user[entry["user"]] += 1
            by_level[entry["level"]] += 1
            active_ips.add(ip)
            if entry["level"] in ERROR_LEVELS:
                error_ips.add(ip)

            if channel not in AUTH_CHANNELS:
                continue
            if channel == "auth":
                streaks.pop(ip, None)  # connexion réussie : série rompue
                continue

            streak = streaks.get(ip)
            if streak is None:
                stamp = entry["timestamp"]
                streak = BruteForceAlert(ip, stamp, stamp)
                streaks[ip] = streak
            streak.attempts += 1
            streak.last_seen = entry["timestamp"]
            streak.users.add(entry["user"])
            if streak.attempts == threshold:
                alerts.append(streak)
                if max_alerts is not None and len(alerts) >= max_alerts:
                    break
        else:
            cycles += 1  # bloc entièrement traité, sans interruption
            continue
        completed = False  # sortie par break : détection interrompue
        break
```

In [21]:
```
complet = analyze(transform(parse_logs(read_multiple_logs(paths))), chunk_size=4)
print(f"Analyse complète : completed={complet.completed}, "
      f"{complet.entries_scanned} entrées, {complet.cycles_completed} cycles terminés par le
else")

arret = analyze(transform(parse_logs(read_multiple_logs(paths))), chunk_size=4, max_alerts=1)
print(f"Arrêt anticipé   : completed={arret.completed}, "
      f"{arret.entries_scanned} entrées lues sur 15, {arret.cycles_completed} cycle terminé")
for alerte in arret.alerts:
    print("  →", alerte)
```

```
Analyse complète : completed=True, 15 entrées, 4 cycles terminés par le else
Arrêt anticipé   : completed=False, 7 entrées lues sur 15, 1 cycle terminé
  → BruteForceAlert(ip='10.0.0.5', first_seen=datetime.datetime(2026, 11, 3, 8, 1, 10),
last_seen=datetime.datetime(2026, 11, 3, 8, 1, 18), attempts=5, users={'root'})
```

7 . Résultats

Exécution complète de `main.py`

Cette cellule appelle `main()` exactement comme `python main.py`, sur les deux fichiers de test de `data/`.

In [22]:
```
import main as point_entree

point_entree.main([])
```

```
[2026-11-03 08:00:01] INFO     alice@192.168.1.10:443/TCP  [auth] {'action': 'login', 'status':
'success'}
[2026-11-03 08:00:05] INFO     bob@192.168.1.11:443/TCP  [event] {'action': 'view', 'page': '/
dashboard'}
[2026-11-03 08:01:10] WARNING  root@10.0.0.5:22/TCP  [auth_failure] {'action': 'login',
'status': 'failed'}  ALERTES: auth_failure:10.0.0.5, blacklisted_ip:10.0.0.5, sensitive_port:22/
TCP
[2026-11-03 08:01:12] WARNING  root@10.0.0.5:22/TCP  [auth_failure] {'action': 'login',
'status': 'failed'}  ALERTES: auth_failure:10.0.0.5, blacklisted_ip:10.0.0.5, sensitive_port:22/
TCP
[2026-11-03 08:01:14] WARNING  root@10.0.0.5:22/TCP  [auth_failure] {'action': 'login',
'status': 'failed'}  ALERTES: auth_failure:10.0.0.5, blacklisted_ip:10.0.0.5, sensitive_port:22/
TCP
[2026-11-03 08:01:16] ERROR    root@10.0.0.5:22/TCP  [auth_failure] {'action': 'login',
'status': 'failed'}  ALERTES: auth_failure:10.0.0.5, blacklisted_ip:10.0.0.5, sensitive_port:22/
TCP, severity:ERROR
[2026-11-03 08:01:18] CRITICAL root@10.0.0.5:22/TCP  [auth_failure] {'action': 'login',
'status': 'failed'}  ALERTES: auth_failure:10.0.0.5, blacklisted_ip:10.0.0.5, sensitive_port:22/
TCP, severity:CRITICAL
[2026-11-03 08:02:00] ERROR    carol@172.16.0.3:8080/HTTP  [error] {'code': 500, 'message':
'Internal Server Error'}  ALERTES: severity:ERROR
[2026-11-03 08:03:00] INFO     alice@192.168.1.10:53/UDP  [raw] DNS query example.com
[2026-11-03 09:00:00] INFO     dave@192.168.1.12:443/TCP  [auth] {'action': 'login', 'status':
'success'}
[2026-11-03 09:01:00] ERROR    admin@203.0.113.7:3306/TCP  [error] {'code': 1045, 'message':
'Access denied'}  ALERTES: severity:ERROR
[2026-11-03 09:01:05] ERROR    admin@203.0.113.7:3306/TCP  [error] {'code': 1045, 'message':
'Access denied'}  ALERTES: severity:ERROR
[2026-11-03 09:02:00] WARNING  guest@198.51.100.4:80/HTTP  [raw] {payload json cassé
[2026-11-03 09:03:00] INFO     dave@192.168.1.12:443/TCP  [raw] message | avec | des pipes
[2026-11-03 09:05:00] INFO     erin@192.168.1.13:443/TCP  [event] {'action': 'logout'}

15 entrées valides après transformation, 8 avec alerte(s)

Entrées analysées : 15 (analyse complète)
Par canal  : {'auth': 2, 'event': 2, 'auth_failure': 5, 'error': 3, 'raw': 3}
Par user   : {'alice': 2, 'bob': 1, 'root': 5, 'carol': 1, 'dave': 2, 'admin': 2, 'guest': 1,
'erin': 1}
Par niveau : {'INFO': 6, 'WARNING': 4, 'ERROR': 4, 'CRITICAL': 1}
IP en erreur       : ['10.0.0.5', '172.16.0.3', '203.0.113.7']
IP actives         : ['10.0.0.5', '172.16.0.3', '192.168.1.10', '192.168.1.11', '192.168.1.12',
'192.168.1.13', '198.51.100.4', '203.0.113.7']
IP actives et blacklistées : ['10.0.0.5']
IP actives non blacklistées : ['172.16.0.3', '192.168.1.10', '192.168.1.11', '192.168.1.12',
'192.168.1.13', '198.51.100.4', '203.0.113.7']
IP blacklistées inactives   : []
Différence symétrique       : ['172.16.0.3', '192.168.1.10', '192.168.1.11', '192.168.1.12',
'192.168.1.13', '198.51.100.4', '203.0.113.7']
ALERTE force brute : 10.0.0.5 - 5 échecs consécutifs (08:01:10 -> 08:01:18), users=['root']

Audit du 2026-11-08T17:07:20 (Python 3.12.3)
  globals : 19 noms (class=3, function=11, module=2, variable=3), 5864 octets
  locals  : 8 noms (variable=8), 568 octets
```

Out[22]: 0

Lecture des résultats.

- 19 lignes non vides sont lues dans les deux fichiers. Parmi elles, 4 lignes invalides sont écartées : une ligne corrompue, un niveau `DEBUG`, une date impossible et un port non numérique. Il reste 15 **entrées valides**.
- 8 **entrées déclenchent au moins une alerte** :
  - les 5 échecs de connexion de `10.0.0.5`, qui cumulent échec d'authentification, IP blacklistée et port SSH 22 ;
  - les 3 erreurs applicatives (codes 500 et 1045).
- Le module 3 détecte une **attaque par force brute sur `10.0.0.5`** : 5 échecs consécutifs sur le compte `root` entre 08:01:10 et 08:01:18.

- Dans `main.py`, la blacklist de démonstration contient `10.0.0.5`. C'est pourquoi l'intersection avec les IP actives vaut `['10.0.0.5']`.

Tests unitaires

In [23]:
```
import io
import unittest

flux_sortie = io.StringIO()
suite = unittest.defaultTestLoader.discover("tests", top_level_dir=".")
resultat = unittest.TextTestRunner(stream=flux_sortie, verbosity=1).run(suite)
print(flux_sortie.getvalue().strip().splitlines()[-3:][0])
print(f"{resultat.testsRun} tests, {len(resultat.failures)} échec(s), {len(resultat.errors)}
erreur(s)")
for fichier in sorted(Path("tests").glob("test_*.py")):
    n = fichier.read_text(encoding="utf-8").count("    def test_")
    print(f"  {fichier.name:<22} {n} tests")
```

```
[ingestion] fichier introuvable, ignoré : /tmp/log-engine/data/absent.log
Ran 79 tests in 0.451s
79 tests, 0 échec(s), 0 erreur(s)
  test_aggregation.py    20 tests
  test_ingestion.py      15 tests
  test_rules.py          26 tests
  test_transform.py      18 tests
```

8 . Répartition du travail

Membre	Contribution
Joan MBALLA	Module 1 (générateurs, parsing, contrat `LogEntry`), module 2 (filter, map, lambda, match-case), intégration continue dans `main.py`, dépôt et environnement
BAYA Youness	Module 3 (set, frozenset, defaultdict, détection de force brute), jeux de test, premier jet du rapport (`notebooks/rapport_module3.ipynb`)
PHILIPPE Mathis	Module 4 (`apply_security_rules` , règles de sécurité, audit `globals()` / `locals()` , export JSON)

Méthode. Le contrat de données a été figé au début (forme du dict, tuple `meta`, itérables uniquement). Chaque module a été développé sur sa branche avec ses propres tests, puis mergé et branché dans `main.py` un par un, ce qui a évité de découvrir les incompatibilités à la fin.

Limites et pistes.

- La seconde passe de `main.py` relit les fichiers : sur un vrai flux réseau impossible à relire, il faudrait un seul parcours qui alimente les deux modules (par exemple avec `itertools.tee` borné, ou des consommateurs combinés).
- La blacklist est codée en dur. Elle pourrait être lue depuis un fichier de configuration.
- La détection de force brute compte les échecs consécutifs sans fenêtre de temps. Une fenêtre glissante (par exemple 5 échecs en 60 secondes) serait plus réaliste.